PA2 — Matrix Transformations and Perspective

Student: Alinur S. | **ID:** 240014 | **Group:** AIB-2401 | **Course:** AR/VR/XR Applications

Variant Parameter	Rule & Calculation	Assigned Value
Assigned Solid	Last digit = 4 (PA1 Solid)	Two-step staircase (60 vertices)
Orbit Period T	6 + last digit = 6 + 4	10.0 s per full orbit
Cube Spin Axis	2nd-to-last = 1 mod 3 = 1	Y-axis [0, 1, 0] (speed 1.2 rad/s)
Orbit Plane	3rd-to-last = 0 mod 3 = 0	Horizontal (around y-axis, radius 2.5)
Camera & FOV	4th-to-last = 0 mod 2 = 0	Eye (0, 2.5, 7), Target (0,0,0), FOV 45°

E1 — Order of Transformations

Model Matrix: The solid's transformation is: $\mathbf{M = R_{orbit} \cdot T \cdot R_{self} \cdot S}$, where S is scale pulse $s(t) = 0.65 + 0.15 \sin(2\pi t/3)$, R_{self} is self-spin about Y at 2.0 rad/s, T is translation along X by radius 2.5, and R_{orbit} is orbit rotation about world Y (T = 10 s).

(a) Swap Orbit Rotation & Translation ($\mathbf{M_a = T \cdot R_{orbit} \cdot R_{self} \cdot S}$):

• *Prediction:* The solid translates to (2.5, 0, 0) after rotating, remaining stationary at x = 2.5 without orbiting the cube.

• *Result:* In Fig. 1(b) (t = 2.5 s), the solid stays at x = 2.5 rather than orbiting behind the cube (Fig. 1(a)).

• *Explanation:* glMatrix post-multiplies matrices, so the last called operation acts on vertices first. In M_a, R_{orbit} acts before translation, rotating vertices around their local centroid rather than the world origin.

(b) Swap Translation & Self-Spin ($\mathbf{M_b = R_{orbit} \cdot R_{self} \cdot T \cdot S}$):

• *Prediction:* Vertices translate to radius 2.5 before R_{self}. Thus, R_{self} rotates the displaced solid around the origin at 2.0 rad/s, producing a rapid orbit without independent self-spin.

• *Result:* In Fig. 1(c), the solid revolves rapidly around the cube at (2.0 + 0.2π) rad/s with locked tidal orientation.

• *Explanation:* Shifting the vertex away from the origin before rotation sweeps it through an orbital circle of radius 2.5.

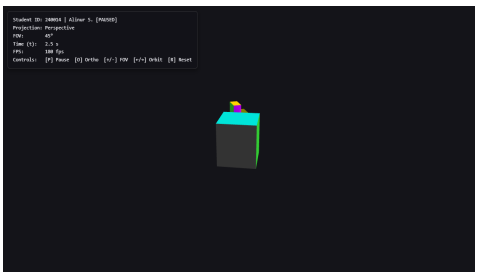


Fig. 1(a): Normal *($R_{orbit} \cdot T \cdot R_{self} \cdot S$)*

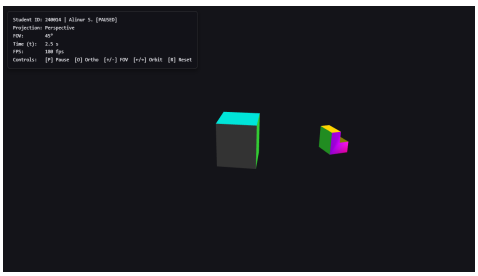


Fig. 1(b): Swap A *($T \cdot R_{orbit} \cdot R_{self} \cdot S$)*

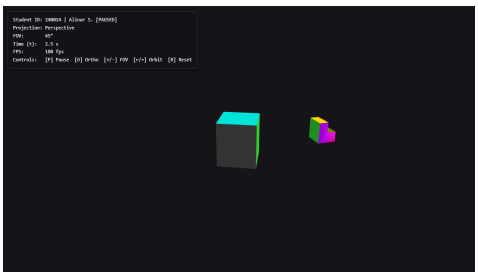


Fig. 1(c): Swap B *($R_{orbit} \cdot R_{self} \cdot T \cdot S$)*

E2 — Three Statements in Chapter 4

(a) mat4.rotate Axis Comment: Part 2 comments *'axis to rotate around (X)'* but passes [0, 1, 0]. **Correction:** [0, 1, 0] is the Y-axis. For X-axis rotation, axis must be [1, 0, 0].

(b) Frustum Clipping vs Face Culling: Chapter 4 labels frustum removal as *'culling'*. **Correction:** Discarding and slicing primitives outside the frustum is **view frustum clipping** (in clip space before perspective divide). **Face culling** (e.g. gl.CULL_FACE) discards backward polygons based on 2D screen winding order.

(c) Homogeneous w vs Quaternions: The chapter links *w* to quaternions. **Correction:** *w* is the projective weight distinguishing points (*w = 1*) from vectors (*w = 0*). Quaternions are hypercomplex numbers representing 3D rotations.

• *Console Test:* Translating by T(5, 10, 15): vec4.transformMat4([1,0,0,0], T) → [1,0,0,0] (vector invariant); vec4.transformMat4([1,0,0,1], T) → [6,10,15,1] (point translated).

• *Explanation:* Translation multiplies *w*: $x' = x + \Delta x \cdot w$. When $w = 0$, translation vanishes because vectors lack position.

E3 — Perspective vs Orthographic Projection

Measured on-screen vertical edge lengths of the cube at paused pose t = 2.5 s (viewport 1280 × 720):

Projection	Nearest Vertical Edge	Farthest Vertical Edge	Ratio (Near / Far)
Perspective (FOV 45°)	117.54 px (depth = 6.900)	103.69 px (depth = 7.966)	**1.1336** (foreshortening)
Orthographic (halfHeight 3.1)	109.36 px (depth = 6.900)	109.36 px (depth = 7.966)	**1.0000** (parallel rays)

Perspective Divide: In perspective, $w_{clip} = -z_{view}$. Dividing by larger w_{clip} at greater depth contracts screen coordinates (7.966 / 6.900 = 1.155, matching the measured 1.1336 ratio). In orthographic, $w_{clip} = 1.0$ everywhere, giving depth-invariant edge lengths (ratio = 1.0000).
Comparison with PA1: PA1's depth offset shifted vertices linearly without depth division, preserving identical face dimensions. Thus, PA1 resembled an **orthographic / oblique parallel projection**.

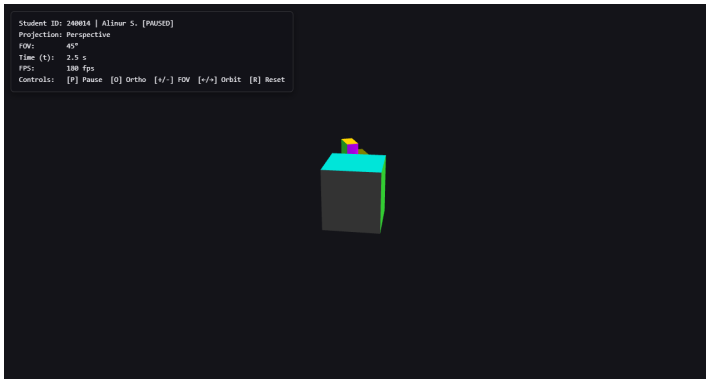


Fig. 2(a): Perspective *(Near = 117.5 px, Far = 103.7 px, Ratio = 1.134)*



Fig. 2(b): Orthographic *(All vertical edges = 109.4 px, Ratio = 1.000)*

E4 — Near, Far and Aspect Ratio

Near & Far Justification: Eye is at (0, 2.5, 7), distance to origin is 7.433. Cube vertices lie within radius 0.866. Solid orbits at radius 2.5 with bounding radius ≤ 0.693. Distance to solid vertices spans $d_{min} \approx 4.24$ to $d_{max} \approx 10.63$. Setting **near = 1.0** and **far = 20.0** ($1.0 < 4.24 \leq d \leq 10.63 < 20.0$) guarantees zero clipping.
(a) Near Plane Cutting Orbit: Setting *near* = *5.08* at t = 7.5 s (when solid depth spans [4.54, 5.62]) slices through the solid (Fig. 3(a)). Vertices closer than 5.08 violate $z_{clip} \geq -w_{clip}$ and are clipped away.
(b) Aspect Hardcoded to 1: Setting *aspect* = *1.0* on a 16:9 canvas (Fig. 3(b)) maps square NDC to a 16:9 viewport, stretching objects horizontally by 1280/720 ≈ 1.78.

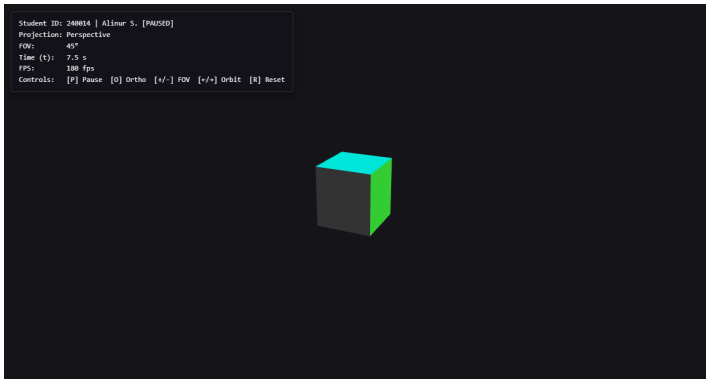


Fig. 3(a): Near Plane Clipping *(near = 5.08 cuts staircase top)*

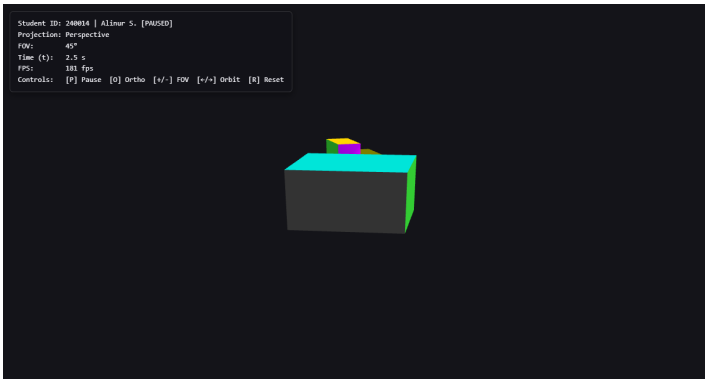


Fig. 3(b): Aspect Distortion *(aspect = 1.0 on 16:9 canvas)*

E5 — Frame-Rate Independence

5-Second dt Benchmark: Sampling 900 frames over 5 s (180 fps baseline): **Mean dt = 5.56 ms**, **Max dt = 5.80 ms**, **Min dt = 5.30 ms**, σ = 0.08 ms.
CPU Throttling: DevTools 4×/6× CPU throttling dropped frame rates to ~30–45 fps (dt ~ 22–33 ms). Driven by continuous time $t = \sum dt$, one orbit took exactly **T = 10.0 s** under all throttle profiles.
dt Clamp Tab-Switch Experiment: Without clamp, switching tabs for 5 s caused $dt \approx 5.0$ *s* on return, jumping half an orbit. With dt = Math.min(now - then, 0.1), resuming advances *t* by ≤ 0.1 s, preventing unnatural jumps.

E6 — One Vertex by Hand

Target Pose: Cube spin axis = Y, $\omega = 1.2$ rad/s. 90° spin occurs at **$t = (\pi/2) / 1.2 = 1.3089969$ s $\approx$ 1.309 s**. Vertex: **$v = (0.5, 0.5, -0.5, 1.0)^T$**.

1. M · v Hand Calculation: 90° Y-rotation (cos 90° = 0, sin 90° = 1):

$M \cdot v = [0(0.5) + 1(-0.5), 1(0.5), -1(0.5) + 0(-0.5), 1.0]^T$ = **$(-0.5, 0.5, -0.5, 1.0)^T$**

glMatrix vec4.transformMat4: [-0.5, 0.5, -0.5, 1.0] (exact match).

2. Clip-Space Result P · V · M · v: $[-0.596868, 1.542777, 6.444762, 7.735737]^T$

3. NDC Hand Calculation (divide by $w_{clip} = 7.735737$):

x_{ndc} = -0.596868 / 7.735737 = **-0.077157** | y_{ndc} = 1.542777 / 7.735737 = **0.199435** | z_{ndc} = 6.444762 / 7.735737 = **0.833115**

Shader Verification: NDC matches WebGL output to 6 decimals: [-0.077157, 0.199435, 0.833115].

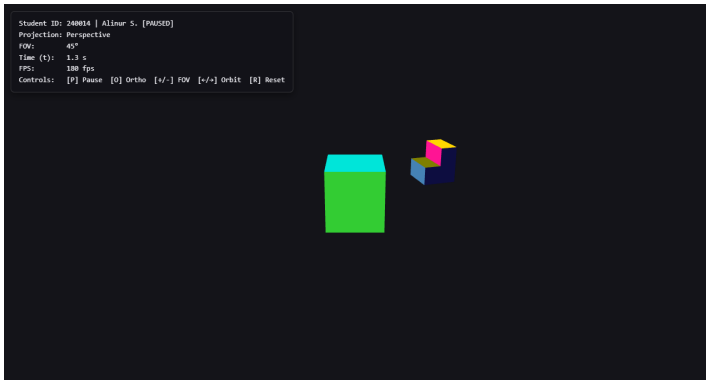


Fig. 4: Cube at 90° Spin Pose *($t = 1.309$ s, showing transformed vertex v_{model})*

Development Log

1. Bug Hit: Rear faces overlapped front faces during orthographic projection mode ([O] toggle).

2. Diagnosis: mat4.ortho near/far values were mistakenly negated, placing far plane in front of near plane.

3. Solution & Commit: Set positive forward distances (near = 1.0, far = 20.0) matching lookAt view space; fixed in commit cdce357.